University of Sheffield

Department of Computer Science

ESP32 IoT Device with Sensor and LED Control via Web Interface

Module: COM3505 – Internet of Things

Professor: Po Yang

Authors: Anurag Dey
Parin Arora

Date: May 8, 2026

Submitted in partial fulfilment of the requirements for COM3505, Spring Semester 2025–2026.

Contents

1 Introduction **2**

2 Hardware Design **2**
2.1 Thermistor Voltage Divider and Temperature Calculation . . . 2
2.2 Hardware Setup . . . 2

3 LED Matrix and Addressability **2**
3.1 Why LEDs Are Not Independently Addressable by Default . . . 2
3.2 Multiplexing: Making the Matrix Effectively Addressable . . . 3
3.3 Implemented Patterns . . . 3
3.4 Text Rendering on the Matrix . . . 3

4 Dual-Core Architecture and Real-Time Rendering **4**
4.1 The Flickering Problem . . . 4
4.2 FreeRTOS Dual-Core Solution . . . 4
4.3 Mutex-Guarded Shared State . . . 4

5 System Architecture **4**
5.1 Data Flow . . . 4

6 Wi-Fi and Flask Web Integration **4**
6.1 ESP32-S3 Wi-Fi and HTTP . . . 4
6.2 Flask API . . . 5
6.3 AJAX Frontend . . . 5

7 Performance Optimisation and Reliability **5**
7.1 RAM and Buffer Efficiency . . . 5
7.2 Timing and Flicker Prevention . . . 5
7.3 Edge Cases and Reliability . . . 5

8 Conclusion **5**

1 Introduction

This report documents the design and implementation of a full-stack Internet of Things (IoT) system centred on an **Adafruit ESP32-S3 Feather** microcontroller. The board uses the **ESP32-S3 mini module**, which integrates the ESP32-S3 system-on-chip: a dual-core 32-bit Xtensa LX7 processor running at up to 240 MHz, 512 KB of SRAM, and an integrated 2.4 GHz Wi-Fi and Bluetooth Low Energy radio. The system reads temperature data from a thermistor, drives a 10×4 multiplexed LED matrix with seven dynamic animation patterns and a scrolling-text mode, and exposes a real-time browser dashboard through a Python Flask server. All inter-component communication occurs over 802.11 Wi-Fi using HTTP/JSON. The dashboard updates continuously via AJAX polling without page refreshes, and allows users to switch LED patterns or transmit arbitrary text to the matrix from any device on the network.

A key architectural decision is the exploitation of the ESP32-S3's dual-core LX7 processor: LED rendering is pinned to Core 1 and networking to Core 0, eliminating the flickering that arises when HTTP calls block a single-core refresh loop. This design decision, and all others, are justified throughout the report in terms of real-time performance and RAM efficiency.

2 Hardware Design

2.1 Thermistor Voltage Divider and Temperature Calculation

The thermistor is read through a simple voltage divider with a 5 kΩ reference resistor between the ADC pin and ground (or VCC, depending on orientation). Given the raw 12-bit ADC reading $V_o \in [0, 4095]$, the thermistor resistance R_2 is recovered from the divider equation:

$$R_2 = R_1 \left(\frac{\mathrm{ADC}_{\max}}{V_o} - 1 \right) \tag{1}$$

where $R_1 = 5\,\mathrm{k}\Omega$ and $\mathrm{ADC}_{\max} = 4095$. Temperature is then computed via the Steinhart–Hart equation, which models the non-linear resistance–temperature relationship of an NTC thermistor as a third-order polynomial in $\ln R_2$:

$$\frac{1}{T} = c_1 + c_2 \ln R_2 + c_3 (\ln R_2)^3 \tag{2}$$

where T is the absolute temperature in Kelvin and the coefficients $c_1 = 1.009249522 \times 10^{-3}$, $c_2 = 2.378405444 \times 10^{-4}$, $c_3 = 2.019202697 \times 10^{-7}$ are the published constants for the specific NTC thermistor used. The result is converted to Celsius ($T_C = T - 273.15$) and finally to Fahrenheit for display:

$$T_F = \frac{9}{5} T_C + 32 \tag{3}$$

2.2 Hardware Setup

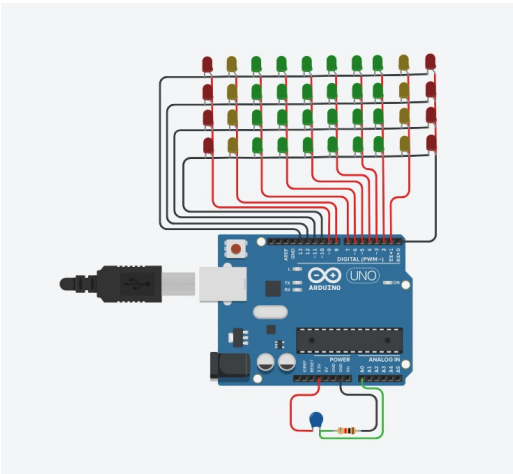

Figure 1: Hardware assembly of the ESP32-S3 IoT system showing the 10×4 LED matrix, breadboard connections, GPIO mappings, and sensor circuitry. *Note: the schematic representation uses an Arduino Uno symbol in place of the ESP32-S3 Feather due to component-library limitations in the diagramming tool; the code is functional after appropriate remapping of pin numbers to the ESP32-S3 Feather pinout.*

3 LED Matrix and Addressability

3.1 Why LEDs Are Not Independently Addressable by Default

In a naive direct-drive design, N LEDs require N GPIO pins. For 40 LEDs this is physically impossible on the ESP32-S3's available IO. A 10×4 matrix reduces pin count to $10 + 4 = 14$ by sharing column (anode) and

row (cathode) lines across all LEDs in the same row or column. However, this sharing produces a fundamental addressing problem.

Consider attempting to light only the two diagonal corner LEDs, $(0, 0)$ and $(3, 9)$, simultaneously. To turn on $(0, 0)$ requires driving column 0 high and row 0 low. To turn on $(3, 9)$ requires driving column 9 high and row 3 low. But once all four pins are in those states, current also flows through the LED at $(0, 9)$ – column 9 is high *and* row 0 is low – and through $(3, 0)$ for the same reason. Four LEDs light up when only two were intended. These unintended pixels are known as "ghost" pixels, and they make any attempt to display arbitrary patterns impossible if more than one row *and* more than one column are driven at the same time.

3.2 Multiplexing: Making the Matrix Effectively Addressable

The solution is **time-division multiplexing**: only one pixel is enabled at any instant, for a fixed dwell time (`PIXEL_TIME_US` $= 40\,\mu$s), then all pins are driven to their off states before the next pixel is activated. Because the refresh cycle ($40 \times 40\,\mu$s $= 1.6$ ms) is far shorter than the human eye's flicker-fusion threshold ($\approx$16 ms for 60 Hz), the display appears fully lit. Since at most one column and one row are ever active simultaneously, ghosting is structurally impossible.

```cpp
void refreshMatrix() {
  for (int c = 0; c < COLS; c++) {        // iterate columns
    for (int r = 0; r < ROWS; r++) {      // iterate rows
      if (getLED(r, c)) {
        allOff();                           // guarantee clean state
        digitalWrite(rowPins[r], ROW_ON); // sink cathode
        digitalWrite(colPins[c], COL_ON); // source anode
        delayMicroseconds(PIXEL_TIME_US); // dwell
      }
    }
  }
  allOff();
}
```

Listing 1: One-LED-at-a-time refresh function (main.cpp).

The `pattern_buf` array acts as a **framebuffer**: pattern algorithms write to it freely, and `refreshMatrix()` reads it without coupling to any specific pattern. This separation mirrors the framebuffer architecture in desktop GPU pipelines and is the primary reason the design can swap patterns instantaneously without display artefacts. The framebuffer is bit-packed to one bit per pixel: 40 LEDs require 40 bits, which fit into 5 bytes (`uint8_t[5]`). A naive `bool` array would consume 40 bytes (since the C++ `bool` type compiles to one byte under GCC), so the bit-packed representation is 8× more memory-efficient. Pixel access is wrapped in `inline` helpers (`getLED`, `setLEDfast`) so the refresh inner loop pays zero call overhead, and `memset` can clear or fill the entire buffer in a single instruction-level operation.

3.3 Implemented Patterns

All patterns write to `pattern_buf` inside the mutex-guarded animation step; `refreshMatrix()` reads from the buffer independently. Frame intervals are fixed constants; switching patterns resets `animStep` to zero and calls `clearPattern()` to eliminate ghost pixels.

Table 1: Implemented LED animation patterns.

Pattern	Interval	Algorithm
`blink`	250 ms	Toggle all 40 pixels between `fillPattern()` and `clearPattern()`
`chase`	250 ms	Single lit column sweeps L→R: `col = animStep % COLS`
`flicker`	250 ms	$\lfloor 40/4 \rfloor = 10$ pixels set at random row/col each frame
`alternate`	250 ms	Checkerboard: `buf[r][c] = ((r+c+step)%2 == 0)`
`snake`	250 ms	Boustrophedon fill: lit pixels accumulate along a bidirectional snake path
`diagonal`	250 ms	Diagonal stripe: `setLED(r, animStep-r, true)` for valid columns
`fill`	–	All 40 pixels on; static
`scroll_text`	250 ms	Pixel-scrolling bitmap font (see §3.4)
`rainbow`	250 ms	Cycles between reg, yellow, green and vice versa
`alert`	250 ms	Wi-Fi like bars with the last bar blinking

3.4 Text Rendering on the Matrix

Characters are encoded as a 3-bit-wide, 4-row-tall bitmap font, stored as a `FontChar` struct array. Each row bitmask uses bits 2, 1, 0 for the left, centre, and right pixels respectively. A full A–Z and 0–9 table is implemented. The scrolling-text algorithm maps a virtual "source column" index (advancing by one each animation frame) to a physical display column using integer division and modulo:

$$\text{charIndex} = \lfloor s/4 \rfloor, \quad \text{charCol} = s \bmod 4 \tag{4}$$

where $s = (\texttt{scrollCol} + \texttt{displayCol}) \bmod (\text{len}(m) \times 4)$ and the factor of 4 accounts for the 3-pixel glyph plus 1-pixel inter-character gap. The algorithm operates in $O(\text{COLS})$ per frame, requires no intermediate scroll

buffer, and wraps seamlessly. Text is rendered into the central six columns (indices 2–7) of the 10-column matrix, preserving a symmetric visual border.

The font table stores each glyph as four 3-bit row masks. `bitRead(p[r], 2-charCol)` extracts the correct pixel, with column 3 acting as a blank inter-character gap. The user types text in the browser, which POSTs to Flask's `/api/set_scroll_text`; Flask sanitises, uppercases, and appends a trailing space before storing it. The ESP32-S3 retrieves the string on the next `/api/get_command` poll and begins scrolling immediately, with the entire pipeline completing within $\leq$2 s end-to-end.

4 Dual-Core Architecture and Real-Time Rendering

4.1 The Flickering Problem

The ESP32-S3 SoC contains two independent 240 MHz Xtensa LX7 cores. In a naïve single-core implementation, `loop()` interleaves LED refresh calls with HTTP transactions. An HTTP POST over Wi-Fi blocks the thread for approximately 80–200 ms (TCP handshake, TLS negotiation, server response). During this window, `refreshMatrix()` cannot execute. With a nominal refresh period of $\approx$1.6 ms, a 100 ms HTTP call causes tens of missed refresh cycles, producing a clearly visible LED blackout that worsens with network latency.

4.2 FreeRTOS Dual-Core Solution

The firmware addresses this by pinning two FreeRTOS tasks to separate cores using `xTaskCreatePinnedToCore`. Core 1 runs `matrixTask` at priority 2, which calls `refreshMatrix()` continuously and advances the animation state every 250 ms. Core 0 runs `networkTask` at priority 1, which executes HTTP POSTs and GETs on their respective intervals. Because the two tasks execute on physically separate processor cores, HTTP blocking *does not preempt* the refresh loop. The result is a stable, flicker-free display regardless of network latency.

4.3 Mutex-Guarded Shared State

The two tasks share `pattern_buf`, `currentMode`, and `scrollMessage`. A FreeRTOS binary mutex (`xSemaphoreCreateMutex`) serialises writes to these structures. Inside `applyCommand()`, the network task calls `xSemaphoreTake` with a 100 ms timeout before mutating shared state and `xSemaphoreGive` immediately afterwards; if the take times out the update is skipped and the same command is reapplied on the next poll.

The matrix task acquires the mutex for at most 5 ms per animation step. The network task waits up to 100 ms; if the mutex is unavailable, the update is deferred to the next poll cycle rather than blocking indefinitely, preventing deadlock. A torn read of `pattern_buf` by the render loop (without holding the mutex) produces at worst one incorrect pixel for one 1.6 ms frame – visually imperceptible – so the render path is deliberately left lock-free to maximise throughput.

5 System Architecture

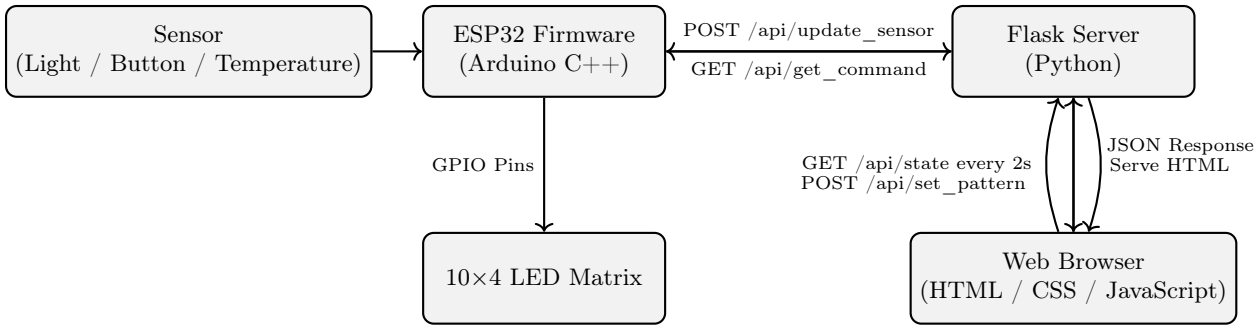

Figure 2: System architecture of the IoT platform showing communication between the ESP32 firmware, Flask server, AJAX-enabled web interface, sensor subsystem, and LED matrix hardware.

5.1 Data Flow

The ESP32-S3's network task POSTs a Steinhart–Hart-derived temperature reading every 3 s to `/api/update_sensor`. Every 2 s it GETs `/api/get_command`, which returns the current pattern name and scroll text. The browser polls `/api/state` every 2 s via AJAX and updates the DOM without page reload. Pattern changes and text submissions are sent as JSON POSTs (`/api/set_pattern` and `/api/set_scroll_text`), propagating to the ESP32-S3 within one polling cycle ($\leq$2 s).

6 Wi-Fi and Flask Web Integration

6.1 ESP32-S3 Wi-Fi and HTTP

Wi-Fi initialisation uses `WiFi.mode(WIFI_STA)` and `WiFi.begin()`, blocking in `setup()` for up to 20 s before handing off to the FreeRTOS tasks. If the link drops, the network task detects `WiFi.status() != WL_CONNECTED` and skips the HTTP call rather than blocking Core 0, which would starve the mutex and indirectly stall the animation. Sensor readings are serialised into a 128-byte `StaticJsonDocument` (avoiding heap allocation), POSTed to `/api/update_sensor` with a 3-second timeout, and the connection is reused across

calls (`http.setReuse(true)`) to reduce TCP overhead.

6.2 Flask API

Flask runs with `debug=True` and is accessible at `0.0.0.0:5000`. All shared state is held in a Python dictionary in RAM. The `/api/get_command` route updates `last_esp_seen` on every call; the `/api/state` handler declares the device offline if no contact has been made in the last 10 s, surfacing a red indicator in the browser. Pattern names are validated against a whitelist (`ALLOWED_PATTERNS`) that mirrors `parseMode()` in the firmware exactly, and the `/api/set_scroll_text` endpoint enforces a 60-character maximum, uppercases the input, and appends a trailing space to ensure a clean visual gap when the message wraps on the matrix.

6.3 AJAX Frontend

The dashboard polls `/api/state` every 2 s using the native `fetch()` API, updating the sensor value, online status, and active pattern badge without reloading. Pattern buttons POST to `/api/set_pattern`; the "Send & Play" control POSTs to `/api/set_scroll_text`. A CSS `@media` breakpoint at 680 px collapses the three-column card grid to a single column, providing a fully functional mobile layout.

7 Performance Optimisation and Reliability

7.1 RAM and Buffer Efficiency

The bit-packed `pattern_buf` occupies just 5 bytes (40 bits) – an 8× saving over a naive `bool` array. `StaticJsonDocument<128>` and `<256>` are stack-allocated, avoiding heap fragmentation. The 64-character `scrollMessage` buffer uses `strncpy` with an explicit null-terminator write, preventing overrun from malicious or oversized server strings. No heap allocation occurs in the matrix task, guaranteeing deterministic execution time.

7.2 Timing and Flicker Prevention

The one-pixel-at-a-time scan at `PIXEL_TIME_US` = 40 μs yields a full-frame period of $40 \times 40\,\mu\text{s} = 1.6$ ms, equivalent to a 625 Hz effective refresh – ten times the flicker-fusion threshold. The dual-core design ensures this rate is maintained irrespective of HTTP latency. If memory constraints demanded a larger matrix (e.g., 32×8), the per-pixel dwell time could be reduced to 10 μs, preserving a 390 Hz refresh on 256 pixels.

7.3 Edge Cases and Reliability

- **Wi-Fi dropout**: The network task checks `WiFi.status()` before each HTTP call and silently skips the call if disconnected, preserving the last-known pattern on the matrix.
- **Invalid thermistor reads**: `readTemperatureF()` returns `NaN` if the raw ADC value is 0 or 4095 (open/short circuit); the network task skips the POST, and Flask retains the previous valid reading.
- **Invalid API inputs**: Flask validates pattern names against `ALLOWED_PATTERNS` and text length ≤60 chars, returning HTTP 400 for violations. The ESP32-S3's `parseMode()` returns `currentMode` for unrecognised strings, maintaining stable operation.
- **Mutex timeout**: If the matrix task holds the mutex for longer than 100 ms (impossible under normal conditions, but guarded against), `applyCommand()` returns without modifying state, and the identical command is retried on the next 2-second poll.

8 Conclusion

The implemented system successfully demonstrates end-to-end IoT integration: a thermistor reading propagates over Wi-Fi to a Flask server and appears on a live browser dashboard within 2 s, while the LED matrix runs eight smooth animation patterns – including pixel-accurate scrolling text – at a stable 625 Hz refresh rate. The dual-core FreeRTOS architecture eliminates flicker by physically separating rendering from networking; the mutex-guarded shared framebuffer prevents data races without introducing latency; and the RESTful Flask API provides clear separation of concerns between firmware, server, and frontend.

Potential extensions include: (1) MQTT to eliminate polling latency; (2) hardware timer ISR for refresh, freeing Core 1 for additional computation; (3) SQLite persistence for long-term sensor history; (4) OTA firmware updates via `ArduinoOTA`. The current architecture scales to multiple ESP32-S3 devices by keying Flask state on device MAC address, requiring only minor server changes.